

Kiến trúc SOA & Microservices

Đặng Ngọc Hùng

Mục lục

1. Lời nói đầu	4
1.1. Tại sao viết sách này?	4
1.2. Về thiết kế không có chương Kiểm thử (Testing) riêng biệt	5
1.3. Độc giả mục tiêu	5
1.4. Cách sử dụng sách	6
1.5. Quy ước trong sách	6
1.6. Bảng tra cứu tài liệu tham khảo	6

1.

CHƯƠNG 1.

Lời nói đầu

Cuốn sách này ra đời từ một nhu cầu thực tế: giúp sinh viên, lập trình viên và kỹ sư phần mềm Việt Nam tiếp cận kiến trúc hướng dịch vụ (SOA) và microservices một cách **có hệ thống, thực tiễn, và dễ hiểu**. Nếu người đọc đã từng bức xúc vì hệ thống đăng ký tín chỉ sập mỗi đầu kỳ học, hay hệ thống LMS nộp bài tập treo hệ thống hoặc quá tải trong những đêm sinh viên nộp bài hạn chót (deadline), thì cuốn sách này, thông qua case study KBM, sẽ chỉ ra cách các kỹ sư phần mềm thực sự giải quyết những vấn đề đó.

1.1. Tại sao viết sách này?

Thị trường đã có nhiều sách có giá trị về microservices — *Microservices Patterns* của Chris Richardson, *Building Microservices* của Sam Newman, *Designing Data-Intensive Applications* của Martin Kleppmann. Tuy nhiên, hầu hết đều viết bằng tiếng Anh và hướng đến developer đã có kinh nghiệm.

Cuốn sách này khác ở ba điểm:

1. **Đứng trên vai người khổng lồ (SOA)** — Mặc dù Microservices đang thịnh hành, cuốn sách không bỏ quên gốc rễ của nó. Các nguyên lý nền tảng của Kiến trúc Hướng Dịch vụ (SOA) như Hợp đồng dịch vụ (Service Contract), Khớp nối lỏng lẻo (Loose Coupling), hay Đóng gói logic (Abstraction) vẫn là cốt lõi. Sách sẽ thể hiện sự tiến hóa từ SOA sang Microservices một cách tự nhiên và có hệ thống.
2. **Case study xuyên suốt mang tính thực tiễn** — thay vì sử dụng các ví dụ rời rạc và “lý tưởng hóa” về mặt học thuật, toàn bộ 12 chương sử dụng chung KBM, một hệ thống LMS (Learning Management System) thực tế đang được vận hành. Giống như việc nâng cấp từ một “Bài tập lớn môn học” (Monolith - nơi mọi thứ gộp chung tất cả thành phần, chỉ cần đáp ứng yêu cầu cơ bản để qua môn) thành một “Hệ thống phục vụ toàn trường” (Microservices), kiến trúc của KBM chứa đựng những “nợ kỹ thuật” (technical debt), những quyết định đánh đổi (trade-offs), và đôi khi là thiết kế chưa thực sự tối ưu. Cuốn sách không cố gắng che giấu điều đó, mà sử dụng chính sự “chưa hoàn thiện” này làm điểm tựa sư phạm: mỗi chương sẽ phân tích *hiện trạng* → *gap* → *migration path* để nội dung sát với thực tế.

3. **Problem-first** — mỗi pattern, mỗi concept bắt đầu bằng “bài toán”: *vấn đề gì? → tại sao monolith không giải quyết được? → microservices giải quyết thế nào?* Đây là cách tiếp cận phù hợp với người học cần hiểu trade-off trước khi áp dụng kỹ thuật.

1.2. Về thiết kế không có chương Kiểm thử (Testing) riêng biệt

Nhiều người đọc có thể thắc mắc tại sao một cuốn sách về microservices lại không dành riêng một chương để trình bày về kiểm thử (Testing). Đây là một quyết định thiết kế có chủ ý của tác giả nhằm không phá vỡ cấu trúc 12 chương chính, dựa trên góc nhìn thực tế của một nhà phát triển và thiết kế hệ thống. Tuy nhiên, để đảm bảo đầy đủ nội dung, sách đã có riêng **Phụ lục G về Testing**. Cụ thể:

Kiểm thử lồng ghép vào ngũ cánh kỹ thuật — Thay vì tách rời kiểm thử thành một phần độc lập, cuốn sách lựa chọn lồng ghép các phương pháp kiểm thử trực tiếp vào các chương tương ứng (ví dụ: kiểm thử hợp đồng (Contract Testing) được đề cập khi thiết kế API ở Chương 3, kiểm thử tích hợp (Integration Testing) và kiểm thử thành phần (Component Testing) khi nói về giao tiếp và chuyển đổi ở Chương 10, hay Chaos Engineering trong chương Observability ở Chương 11).

Tập trung vào tư duy kiến trúc và thiết kế — Mục tiêu chính của cuốn sách là trang bị cho người học tư duy thiết kế hệ thống, phân rã dịch vụ và xử lý dữ liệu phân tán. Các kỹ thuật viết test case chi tiết thường phụ thuộc nhiều vào ngôn ngữ lập trình và framework cụ thể. Do đó, các bài tập thiết kế kiểm thử được phân bổ trực tiếp vào danh sách bài tập ở cuối sách dưới dạng các tình huống xử lý lỗi và thiết kế kịch bản kiểm thử thực tế.

1.3. Độc giả mục tiêu

Developer ở mức junior/mid-level — đang làm việc với monolith và muốn hiểu sâu sắc nguyên lý thiết kế dịch vụ (SOA) cũng như cách triển khai qua kiến trúc microservices — đây là đối tượng mà cuốn sách lấy làm trọng tâm khi cân nhắc độ sâu và nhịp trình bày.

Sinh viên CNTT năm 3-4 — muốn hiểu kiến trúc phần mềm hiện đại — và cuốn sách được viết với niềm tin rằng người đọc hoàn toàn có thể hấp thụ được độ sâu này.

Kỹ sư phần mềm — cần tài liệu tham chiếu có hệ thống về patterns và practices

Trưởng nhóm kỹ thuật — cần đánh giá khi nào nên/không nên dùng microservices

Một lời với các sinh viên và developer mới vào nghề: cuốn sách này *không* hạ thấp độ sâu kỹ thuật để “dễ đọc hơn”. Trong thời đại AI và bùng nổ thông tin, ranh giới giữa “kiến thức cho người mới” và “kiến thức cho người có kinh nghiệm” đang mờ đi rất nhanh — chúng ta hoàn toàn có thể, và nên, tiếp cận những kiến thức sâu ngay từ sớm. Có thể ở lần đọc đầu, một vài phần sẽ cần người đọc chịu khó tra cứu và đọc lại; điều đó là bình thường và đã được lường trước. Mục tiêu là để khi bước vào những dự án thực tế sau này, các khái niệm ở đây không còn xa lạ mà trở thành thứ *đã từng gặp* và giờ mới thực sự *hiểu sâu sắc*. Các “công cụ hỗ trợ” — box “Ôn lại kiến thức nền”, marker “nâng cao, có thể đọc lướt lần đầu”, và Phụ lục A (Bảng thuật ngữ) — được đặt rải rác chính là để đồng hành cùng người đọc trên quá trình đó.

Yêu cầu kiến thức nền:

- Lập trình Java và Spring Boot ở mức **đã từng tự tay viết một ứng dụng CRUD** có JPA và REST controller (không cần thành thạo, nhưng cần quen với vòng đời cơ bản của một service Spring).
- Hiểu REST API, SQL, HTTP; có nền tảng cơ sở dữ liệu cơ bản (transaction, isolation level ở mức khái niệm).
- Biết dùng Git, Docker (ở mức cơ bản).

- Quen đọc thuật ngữ kỹ thuật tiếng Anh — sách giữ nguyên nhiều thuật ngữ tiếng Anh; có **Phụ lục A (Bảng thuật ngữ)** với định nghĩa ngắn gọn để tra khi cần.

1.4. Cách sử dụng sách

Đọc tuần tự: Sách thiết kế theo logic tiến trình — từ khái niệm (Phần I) → giao tiếp & dữ liệu (Phần II) → hạ tầng & vận hành (Phần III). Mỗi chương xây trên kiến thức chương trước.

Tra cứu: Phụ lục C (Pattern Catalog) liệt kê 45+ patterns với chương tham chiếu để tra cứu nhanh khi cần.

Case study: Theo dõi KBM xuyên suốt sách để thấy một kiến trúc thực tế tiến hóa từ monolith sang microservices.

1.5. Quy ước trong sách

Ký hiệu	Ý nghĩa
[RULE] Nguyên tắc	Triết lý hoặc best practice quan trọng
[CASE] Phân tích gap	So sánh KBM hiện trạng vs best practice
! Sai lầm thường gặp	Anti-patterns và cách phòng tránh
[Tip] Tip	Mẹo thực hành
[Note] Ghi chú	Thông tin bổ sung, lưu ý quan trọng cho độc giả
[2a, Ch.3]	Tham chiếu đến sách (mã trong bảng tra cứu bên dưới)

1.6. Bảng tra cứu tài liệu tham khảo

Để thuận tiện cho việc tra cứu và đối chiếu kiến thức giữa các nguồn tài liệu kinh điển, cuốn sách sử dụng hệ thống mã hóa rút gọn cho các đầu sách tham khảo chính:

Mã	Tên sách	Tác giả	Phiên bản
[1]	SOA Analysis & Design for Services and MS	Thomas Erl	2nd Ed (2017)
[2a]	Microservices Patterns	Chris Richardson	1st Ed (2018)
[2b]	Microservices Patterns	Chris Richardson	2nd Ed (2025)
[3]	Microservices: Up and Running	Ronnie Mitra & Irakli Nadareishvili	1st Ed (2020)
[4a]	Building Microservices	Sam Newman	1st Ed (2015)
[4b]	Monolith to Microservices	Sam Newman	1st Ed (2019)
[5]	Practical Event-Driven MS Architecture	Hugo Rocha	1st Ed (2022)
[6]	Domain-Driven Design	Eric Evans	1st Ed (2003)
[7]	Designing Data-Intensive Applications	Martin Kleppmann	1st Ed (2017)

Bảng P.1: Hệ thống mã hóa tài liệu tham khảo xuyên suốt sách